

# University of Papua New Guinea

## 1.40823 Object Oriented Programming

### Tutorial 6: Abstraction and Interface (System Interface)

#### Exercise 1: Steps for Changing Banking\_Interface Package

Download new BankingSystem project from moodle under Abstraction and Interface topic. Change the package Banking to Application Interface. The Steps are:

1. Right-click on banking\_interface
2. Select Refactor
3. Select Rename
4. Replace banking\_interface to application\_interface
5. Click OK

#### Exercise 2: Create System\_Interface Package

To create the package, follow these steps:

1. Right-click on BankingSystem → New
2. Select Package
3. Type System\_Interface → Finish

Into this package, create the following abstract class and interfaces.

#### Exercise 3: Banking Interface BankAccount

Write an interface declaration called BankAccount with the following method declarations. Save the interface.

```
public abstract void deposit(double amount);
public abstract void withdraw(double amount);
public abstract void showBalance();
public abstract double retrieveBalance();
public abstract void applyInterest();
```

#### Exercise 4: Redefine Banking Class Account Implements BankingAccount

Add to the declaration of the class Account to implement the BankAccount interface. Override the interface by writing the method definitions.

#### Exercise 5: Implement Interface BankSystem

Write an interface declaration called BankSystem with the following method declarations. Save the interface.

```
public abstract void addAccount(Account account);
public abstract void removeAccount(Account account);
public abstract void showAccounts();
public abstract void showAccountDetails(int accountNumber);
public abstract Account searchAccount(int accountNumber);
public abstract List<Account> retrieveAccountList();
```

Add another package called System\_Implement. Add new class BankSystemIm to implement this interface.

In the BankSystemIm class, add a data member (field) called acctList of Account List. This will contain a list of accounts.

The methods implemented must be based on Account objects stored in the list. Before you implement the class fully, the class constructor must be implemented with the list object created in this implementation.

### Exercise 6: Abstract Class Bank

Write an abstract class called Bank under the System\_Interface package. In the abstract class, the following will be its content.

```
package system_interface;

import java.util.ArrayList;
import java.util.List;
import banking.Account;

public abstract class Bank {
    protected List<Account> acctList;           // list of accounts
    public Bank() {
        acctList = new ArrayList<Account>();
    }
    public void addAccount(Account account) {
        acctList.add(account);
    }

    public abstract void removeAccount(Account account);
    public abstract void showAccounts();
    public abstract void showAccountDetails(int accountNumber);
    public abstract Account searchAccount(int accountNumber);

    public List<Account> retrieveAccountList(){ return acctList; }

    public void LoadAccountList(List<Account> accountList) {
        for(Account acc : accountList) {
            acctList.add(acc);
        }
    }
}
```

### Exercise 7: Class BankIm Extends Abstract Class Bank

The Bank class has other implementations completed but there are four methods that are still in abstract form and they require implementations. This will be achieved in the System\_Implement package under the class name BankIm.

The removeAccount method removes an account from the list.

The showAccounts method lists or prints available accounts

The showAccountDetails method passes an account number and prints the details of a particular account matching the account number.

The searchAccount method searches an account by the account number passed and returns the account matching the number.